

TourPlanner

Intermediate Protocol

1. Application Overview

TourPlanner is a web-based tour management application. Users register and log in, then create and manage tours across five transport types: BICYCLE, RUNNING, and HIKING. For each tour, users can record multiple tour logs with statistics from accomplished tours.

Technology Stack:

Frontend	Angular
Backend	Spring Boot with layered architecture (Controller -> Service -> Repository)
Database	PostgreSQL via JPA/Hibernate

2. UI Flow

The application consists of four main screens connected as follows:

- Login / Register - Entry point. Toggle between login and registration mode.
- Dashboard - Central hub. Lists all tours, provides name search, and buttons for creating new tours.
- Tour Detail - Shows all tour attributes, a route map, and the list of tour logs with inline add/edit/delete.
- Tour Form - Reused for both creating and editing tours. Contains validated input fields and a map preview.

3. Wireframes

3.1 Login / Register Screen

A centered card with the application logo, form fields, and a toggle link to switch between login and registration mode. Registration adds „Name“ and „Confirm Password“ fields.

LOGO

Tour Planner

Login to your account

EMAIL

PASSWORD

LOGIN

Don't have an account? [Register](#)

3.2 Dashboard Screen

Shows the navbar at the top, an action bar with search and buttons, and a vertical list of tour cards. Each card contains a map preview thumbnail, tour name, route (from -> to), transport badge, and a stats grid (distance, logs, popularity, child-friendliness).

LOGO

Tour Planner

☰

🔗

+ NEW TOUR

📶 IMPORT

📄 EXPORT

MY TOURS (4)

[MAP PREVIEW]

test

📍 Josef-Frank-Gasse 4/12 → rennweg 89 ⌚ 5h 0m

Hiking

DISTANCE
15.0 km

LOGS
1

POPULARITY
Low

CHILD-FRIENDLY
Yes

[MAP PREVIEW]

test2

📍 test → tes2 ⌚ 5h 0m

Hiking

DISTANCE

LOGS

POPULARITY

CHILD-FRIENDLY

3.3 Tour Detail Screen

Displays all tour attributes, a route map (Leaflet), computed statistics, and the full list of tour logs. Logs can be added, edited, and deleted inline via a collapsible form.

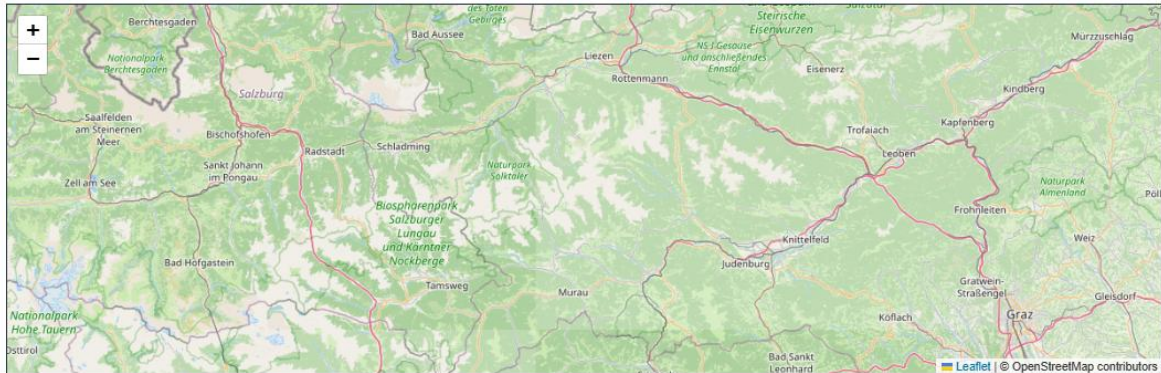
[← BACK](#)[EDIT](#)[DELETE](#)

test

Hiking

test

⌕ Josef-Frank-Gasse 4/12 → rennweg 89 ⌚ Estimated: 5h 0m

DISTANCE
15.0 kmEST. TIME
5h 0mPOPULARITY
Low
(from 1 logs)CHILD-FRIENDLY
Yes
(computed)

TOUR LOGS (1)

[+ ADD LOG](#)

📅 03/09/2026 at 18:11 #1

daf1

Difficulty: MEDIUM

Distance: 11.0 km

Time: -

Rating: ★



3.4 Tour Create / Edit Form

A single form component reused for both creating new tours and editing existing ones. Organized into sections: Basic Information, Route Information (with transport type selector), Distance & Time, and a live map preview. Required fields are marked with * and validated before submission.

CREATE NEW TOUR

BASIC INFORMATION

TOUR NAME *

Required field

DESCRIPTION *

Required field

ROUTE INFORMATION

FROM (START LOCATION) *

TO (END LOCATION) *

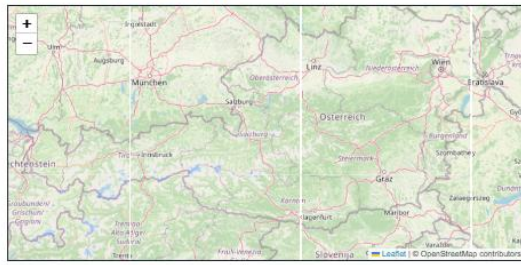
TRANSPORT TYPE *

DISTANCE & TIME

TOUR DISTANCE (km)

ESTIMATED TIME (seconds)

ROUTE MAP PREVIEW



VALIDATION

- All fields marked with * are required
- Start and end locations must be valid addresses
- Tour name must be unique
- Form validates input before submission

4. Reusable Components

4.1 NavbarComponent (shared)

A shared navigation bar displayed on all authenticated pages. Contains the application logo, title, a menu icon, and a logout button. It is imported as `<app-navbar>` in the Dashboard and Tour Detail/Form views.

4.2 TourMapComponent (shared)

A reusable map component wrapping Leaflet.js. Accepts `@Input()` properties for `routeGeoJson`, `fromLocation`, and `toLocation`. Used in both the Tour Detail view (to display the full route) and

the Tour Form (as a live map preview). Handles map initialization, GeoJSON rendering, and automatic bounds fitting.

5. Data Binding & Reactive State

The application follows the MVVM (Model-View-ViewModel) pattern with zoneless change detection. Each feature has a dedicated ViewModel class that manages signal-based reactive state, keeping components as thin views:

- Model layer: `core/models.ts` defines the Tour, TourLog, and User interfaces. Services (TourService, AuthService) handle HTTP calls and return typed Observables consumed only by ViewModels.
- ViewModel layer: dedicated `*.viewmodel.ts` classes (LoginViewModel, DashboardViewModel, TourDetailViewModel, TourFormViewModel) hold all view state as Angular signals. Components inject their ViewModel via `inject(XxxViewModel)` and templates bind to `vm.*` properties.
- View layer: Templates use one-way binding with signal setters (`[ngModel]="vm.field()"` (`ngModelChange`)=`"vm.field.set($event)"`), interpolation (`{{ }}`), and structural directives (`*ngFor`, `@if`) for reactive data display. Classic two-way `[(ngModel)]` is intentionally avoided to preserve signal integration.
- Change detection: The application uses zoneless change detection (`provideZonelessChangeDetection()` in `app.config.ts`). No `ChangeDetectorRef` is used anywhere; UI updates are driven entirely by Angular signals in the ViewModel layer.

6. Responsive Design

The UI adapts to different screen sizes using CSS media queries at the 640px breakpoint:

- Dashboard: Stats grid collapses from 4 columns to 2; map previews expand to full width.
- Tour Detail: Stats grid (4->2 cols), log form grid (2->1 col), log stats (4->2 cols).
- Tour Form: Location fields and distance/time fields stack vertically; transport type grid (4->2 cols).
- Flexbox with flex-wrap and CSS Grid with auto-fit / minmax() ensure fluid layouts.

7. Input Validation

- Tour Form: Uses Angular's NgForm with required attributes. Submission is blocked when the form is invalid. Invalid fields display red borders and „Required field“ hints. Server-side errors are shown in an alert banner.
- Tour Log Form: All fields use one-way `[ngModel]` binding with signal setters. Default values are applied for missing fields (e.g., current timestamp for `dateTime`, rating defaults to 1). Backend validates with Jakarta `@NotNull`, `@Min`, `@Max` annotations.
- Login / Register: Fields are marked required. Backend validates email format, username length (3–50), and password length (min 6).